

Performance- and Energy-Aware Gait-Based User Authentication With Intermittent Computation for IoT Devices

Pavlos Zouridakis[✉], *Graduate Student Member, IEEE*, and Sai Manoj Pudukotai Dinakarrao[✉], *Member, IEEE*

Abstract—Internet of Things (IoT) makes it possible to utilize a multitude of small- and medium-scale devices allowing for increased functionality in flexible networks. However, user authentication on IoT devices is as important as it can be challenging. Due to limitations on the available energy, interface, and processing power, IoT devices can be the weakest link in their networks. Multiple authentication techniques have been developed to address these challenges. However, the existing techniques are limited in terms of performance, overheads, and efficiency. In contrast, our proposed authentication method uses a user's walking gait as an input, because gait is unique to every user and can be collected using low-power inertial sensors found on all handheld devices. Our authentication method uses a lightweight neural network (NN) which is further complimented with early exits to further optimize computational cost(s). We also propose reinforcement learning that considers the energy consumption to dynamically determine which of the exits should be chosen to strike a balance between performance and computational and energy cost. Though effective, one of the challenges with IoT devices is their power supply such as dependence on batteries. Discharge of the battery or any other interruptions can lead to recomputations, which are expensive on already limited battery-operated IoT devices. To address such challenges, especially for user authorization, we introduce intermittent computation to our proposed authentication framework. Intermittent computation can store the state of the NNs at checkpoints. In the case of power disruption, the execution will be resumed from the saved checkpoints instead of performing the whole execution. Most implementations of intermittent computation take place at the compiler level, which makes for a very efficient design, however, that also makes them hardware specific. Our method is implemented on software and is hardware-agnostic, allowing us to create checkpoints to save and retrieve the authentication framework state, in case of power interruption. The proposed authentication framework can authenticate users with up to more than 85% accuracy and can save up to 34% of computations due to the proposed intermittent computing.

Index Terms—Early-exit neural networks (EENets), gait-based authentication, intermittent computing, Internet of Things (IoT) networks, reinforcement learning, user authentication.

Manuscript received 15 September 2022; revised 1 March 2023 and 13 July 2023; accepted 17 August 2023. Date of publication 7 September 2023; date of current version 22 January 2024. This work was supported by the Defense Advanced Research Projects Agency (DARPA) under Agreement HR001120C0154. This article was recommended by Associate Editor Q. Zhu. (Corresponding author: Pavlos Zouridakis.)

The authors are with the Department of Electrical and Computer Engineering, George Mason University, Fairfax, VA 22030 USA (e-mail: pzourida@gmu.edu; spudukot@gmu.edu).

Digital Object Identifier 10.1109/TCAD.2023.3313097

I. INTRODUCTION

OVER the last few years, with the advancements in wireless communication technologies and capabilities (e.g., WiFi, Bluetooth, and 5G), the Internet of Things (IoT) has grown considerably [1] to support different applications and services [2], [3]. As a result, IoT networks have grown to include a variety of devices and protocols. Such heterogeneous environments present important yet unique challenges in maintaining security, data integrity, and confidentiality [4]. To overcome such challenges, it is important to be able to design entire IoT networks at a high-level [5], while understanding the specific challenges of working with smaller IoT devices [6].

IoT devices often come with significant limitations in terms of processing resources, memory, availability of power supply, and user interface. As a result, they become even more vulnerable to threats, such as spoofing and power depletion attacks, among others [7], [8]. To ensure data integrity, reliability, user-specific experience, and resilience to external threats, user authentication in IoT devices is pivotal.

There have been many approaches to deal with these challenges, including cryptographic techniques [9], [10], [11], the use of a password or personal identification number [12], and two-factor authentication [13]. These approaches are computationally expensive and require a specific interface to provide input, such as a keyboard. In addition to that, they are vulnerable to providing the wrong credentials, and repeating the authentication process, puts further stress on the device's limited resources. Other authentication techniques biometric inputs from users, such as voice recognition [14], fingerprint scans [15], or retina scans [16]. Such solutions are not appropriate for mid-scale or low-scale IoT devices, as they require a lot of computations and the deployment of specialized equipment to obtain the user input. To mitigate the computation overhead(s) of such techniques, the solution that involves cloud-based off-loading [17]. However, such techniques incur significant communication costs and they are also susceptible to security attacks [18].

To address the issues mentioned above, we intend to use a biometric input, that is unique to the user, easy and reliable to collect, unobtrusive to the user [19], and robust, i.e., difficult to spoof [20]. Considering these factors, we introduce a user authentication method that utilizes a user's walking gait (or form) [21]. Gait-based authentication can be performed

TABLE I
COMPARISON OF GAIT-BASED AUTHENTICATION AND RSA

Algorithm	Accuracy	Immune to hacking	Continuous Authentication	Passive Input Collection
RSA	100%	✗	✗	✗
ceenet-lite+dqn	85.66%	✓	✓	✓

indirectly (using external images of an individual) or directly (using the user's metrics) [22]. A user's movement at a low level, the way that they contact the floor surface and apply force to it, are unique identifiers [23] and that information can be obtained sufficiently with a wrist-worn device [24] or handheld devices [19]. Our preliminary analysis of gait signals between different users in our training dataset that has over 65 000 samples has shown an average correlation of no more than 0.1447. Such a low correlation suggests only a weak linear relationship between the two users. This indicates that the gait signatures are unique and can effectively be used for user authentication. A user's gait can be acquired by inertial motion sensors, which are already available on every handheld or wrist-worn device. Existing gait-based user authentication techniques require the use of deep neural networks (DNNs). A combination of convolutional and recursive neural networks (RNNs) is used in [19] with good accuracy, but it is very expensive in terms of computations and memory usage. In general, DNNs have gotten better at classification and produce high accuracy in pattern recognition problems, and for that reason, they are used for biometric authentication methodologies. In pursuit of high accuracy, DNNs have gotten bigger, more complex, and more computationally expensive. To address that, recent research has looked to reduce the amount of operations and memory required through pruning [25], quantization [26], [27], and other techniques [28], [29]. Also, in many cases, DNNs are able to extract the necessary features for classification from a small subset of the layers available. For that reason, early termination [30], [31], [32], [33], [34], [35], [36] techniques have been introduced. Considering this, we deploy early-exit neural networks (EENets) for gait-based user authentication [37]. However, traditional EENets are limited by static exit selection which is not effective when different signals require different exits for efficiency (Table I).

On the other hand, IoT devices are deployed outside of the power grid and rely on batteries or energy harvesting. As a result, it is possible that the execution of code that is running on devices in such settings gets interrupted, when the power supply is exhausted. Power depletion attacks describe cases when malevolent users force a device to run code that will exhaust the device's power source, rendering it incapable of performing any other tasks. In such cases, on restarting, the device has to redo the application execution (authentication), which can lead to redundant computations. Recent work enables intermittent computation, through task-based computing [38], checkpointing [39], [40], or aiming to reduce the energy cost of checkpointing [41], [42]. It is, therefore, important to be able to save partial work, so that the energy spent up to the point of interruption is not wasted; also in order to avoid nontermination scenarios as well as power depletion attacks. Nontermination refers to a situation where

a device keeps running code but it can never get to complete the computations required to run the code in its entirety.

To address these challenges, we make the following cardinal contributions.

- 1) We introduce EENets [43], in conjunction with performance-awareness intermittent computing for user authentication. EENets are a class of neural networks (NNs) that are trained on both precision and cost. When a certain confidence threshold is met, computations terminate early. That leads to considerable savings in computational cost.
- 2) We complement the EENet with a deep quality-learning network (DQN) for dynamic exit selection of EENets. The DQN is a reinforcement learning algorithm that can be trained to choose a specific exit from the EENet, in case it is deemed beneficial to do so.
- 3) Our approach was tested on the whuGAIT dataset [19] and it achieves >85% authentication accuracy with nearly $5.9\times$ lower computations compared to traditional DNNs.

II. BACKGROUND

Due to the limitations on processing resources, power supply, memory, and connectivity constraints inherent to low-power IoT devices, we focus on techniques that allow for lightweight authentication. We present some of the relevant works on lightweight authentication methods, early termination, and intermittent computing in this section.

A. Lightweight Authentication Techniques

The authentication methods that are being used on modern applications and computer systems were developed to achieve security without restrictions in processing power. As processors got faster and throughput in networks for higher, the authentication algorithms and cryptographic schemes got more complex and computationally expensive. With the rapid increase of low-power devices, there is a need for smaller computational cost, without sacrificing in security.

Gupta et al. [44] proposed a lightweight anonymous user authentication and key-establishment scheme for wearable devices. It is based on simple XOR and one-way cryptographic hash function to share the session key between the mobile terminal device and the wearable sensing device.

Banerjee et al. [45] proposed a hardware solution for data-gram transport layer security (DTLS) protocol. That includes reconfigurable hardware accelerators, a DTLS engine offloads control flow to hardware and on-chip RISC-V core. The proposed solution results to two orders of magnitude in energy savings and $10\times$ reduction in memory usage. However, that would require widespread of deployment of these dedicated chips on IoT. Also, it may achieve good security, but it does not offer continuous authentication.

The National Institute of Standards and Technology (NIST) has initiated an evaluation process to select lightweight cryptographic standards for constrained environments [46]. All participated algorithms were evaluated on software and hardware there are currently ten finalists, from which the new

standards will be selected. The finalists include Sparkle [47], Romulus [48], and Xoodiak [49]. However, these techniques, though lightweight, involve complex operations, require key storage, and cannot facilitate continuous user authentication.

B. Early Termination

DNNs have become a common application because of the high accuracy they provide in pattern recognition and classification. However, a lot of the increase in accuracy is achieved at the cost of resource consumption and computational cost. For that reason, a lot of recent work has tried to address this issue and allow for faster and more lightweight inference implementations.

Most recent work has sought to develop techniques for model compression that would speed up a deep network through knowledge distillation [50], [51], filter pruning [25], quantization [26], and compression with structured matrices [52]. However, these methods are nondynamic and the same number of computations get applied to all inputs. To achieve dynamic inference, that will differ depending on the input, early termination methodologies have been developed.

Branchynet [32] is a DNN early-termination architecture that uses side branch classifiers. It uses a statistical approach to determine whether it has extracted sufficient features to make a prediction and uses entropy as the confidence function. However, they do not take computational cost into consideration to determine which branch to take.

Panda et al. [31] introduced conditional deep learning (CDL) by using linear classifiers to each convolutional layer and checking the output to determine when inference for a given sample can be exited early. EENets train all exits at the same time and allow for a more general network structure, whereas CDL uses a classifier after every hidden layer.

Wang et al. [36] proposed SkipNets, which are dynamically routed networks that determine which of the layers of a CNN should be included during inference, and the rest of the layers get skipped. To select or exclude a layer, small gating networks are interposed between layers, which map the output of the previous layer to a binary decision to execute or bypass the following layer. Reinforcement learning is used to train the network to choose which layers to include or exclude for the calculation, where there is a reward function for the use of each gate in the gating network.

Beretizshevsky and Even [30] proposed an early termination with cascaded inference similar to [32] and [36]. It utilizes a confidence measure based on a softmax classifier. The training process calculates the convolutional weights together with the last fully connected layers and then begin to optimize the weights of the following fully connected layers.

Adanets [35] use the same premise as SkipNets, with the difference that they use a nonlinear function of two fully connected layers connected with a ReLU activation function to determine the relevance score for each layer. Then they introduce soft constraints for each layer during training, so that the network learns which layer to use in each case.

Figurnov et al. [33] proposed spatially adaptive computation time (SACT) that can be applied to residual networks. It can

focus on a particular spatial position in an image and it can maintain the alignment between the image and the feature space. SACT focuses the computation on regions of the input that are of most interest and ignores other areas to reduce the total number of computation. It works well with RNNs and ResNets but it does not work with any kind of NN.

Wu et al. [34] introduced Blockdrop that tried to minimize the number of ResNet blocks used without decreasing accuracy by using policy search methods from reinforcement learning. The policy network uses a single-step Markov decision process to examine every possible path. Then, policy gradient is used to train for the optimal path.

However, the works above come with certain limitations regarding their use for lightweight authentication. For instance, [30], [31], [32], [33], [35], and [36] only use accuracy for their loss function during training, whereas EENets also consider cost. Also, [34], [35], and [36] only work well with ResNets, and while [33] works with RNNs along with ResNets, we want to use smaller NNs, that are more fit to use on lightweight devices. In addition to that [30] and [31] have a higher overhead for training. In addition to the methodologies described above, Wu et al. [53] developed a methodology that implements early-exit selection while keeping track of the energy available to the system to avoid nontermination. Unlike EENets, their implementation always uses the latest exit possible, consuming the largest amount of energy possible. EENet on the other hand, only moves to a later exit, if it determines that the benefit to accuracy justifies the additional cost.

There are also techniques recently developed specifically for small devices that operate with renewable energy, such as Zygarde [54] and ePerceptive [55]. Zygarde can reduce the size of the DNN that it runs based on the quality of the input signal. Unlike EENet, Zygarde's reduced-sized DNN does not produce a classification decision but sends its result to a k-Means classifier that completes the classification. That approach introduces further overhead. On the other hand, ePerceptive [55] introduces a scheduler to partition the DNN into small tasks, estimates the remaining energy in its system, and determines which tasks are to be executed under the available power budget. The execution of tasks is driven by power budget rather than performance, which makes the execution perform unnecessary computations if the power budget remains available. By comparison, EENet uses both computational cost and accuracy in its loss function to achieve the optimal balance of efficiency and high performance. Notably, neither of those techniques offers an intermittent computing solution that allows them to recover the state when execution terminates unexpectedly.

C. Intermittent Computing

When a device is powered by battery or power harvesters, it is prone to interruptions to code execution when its power source is exhausted. Since power may be available intermittently, it is important that the state of the processor is saved to nonvolatile memory (NVM), so that execution can be resumed when enough energy is available again.

Maeng et al. [38] introduced Alpaca, an extension for low-overhead intermittent computing that does not require checkpointing, using a task-based execution model using a data privatization scheme built on idempotence analysis. It is a single-task execution scheme that does not allow parallelization or concurrent task sequencing.

Chinchilla [40] introduces a compiler and runtime system designed to create checkpoints and dynamically choose which ones will be used, to reduce overhead while running C on energy-harvesting devices. Chinchilla first overprovisions for checkpoints in the code to avoid nontermination and then, based on available energy levels disables checkpoints to avoid execution time overheads due to excessive checkpointing.

In TotalRecall [41], they focus on being able to make quick and frequent checkpoints. When utilizing renewable sources of energy, checkpoints often need to be created frequently, but also the interruptions are short and execution can resume quickly. Saving state to Flash memory takes a long time and cannot be justified for such applications. SRAM on the other hand, although volatile, can hold its charge long enough, so they have developed a library that allows checkpointing on SRAM, but the method is hardware-specific to MSP430 microcontrollers.

To reduce the energy cost of checkpointing Ahmed et al. [42] proposed DICE, a software-based technique that operates in volatile memory to determine differentials. It tracks changes in main memory to only update the affected slices of the existing checkpoint data.

Van Der Woude and Hicks [39] introduced Ratchet, an automatic checkpointing system. It is compiler-automated and it is specific to ARM-based processor architectures. It works by automatically inserting a checkpoint between every write-after-read (WAR) dependency.

Colin and Lucia proposed [56] CleanCut, which estimates the energy cost of arbitrary code, to aid with the placement of checkpoints. First, the code is decomposed to tasks. CleanCut calculates the energy cost of each task and also uses a statistical model for the energy requirements of different paths of tasks through the program. However, this is an error-prone process.

The scope of the implementations in the existing literature on intermittent computing, however, is limited and hardware-specific. The implementations presented above are hardware-specific [38], [39], [40], [41], [56] and they are developed for specific versions of microprocessors. Even [42] that is a software-based solution, could not be used on its own, but instead runs on top of one of the other techniques, adding some overhead of its own. That would make it impossible to deploy any of the aforementioned methodologies on devices with different architectures, where we might want to run gait-based authentication. In addition to that limitation, [41] relies on harvested energy to get collected quickly enough that the checkpoint saved on volatile memory does not get permanently lost; [39] only works with sequential processing and creates too many checkpoints; [40] has some information loss during setup and high overhead during undo logging; [38] requires extensive configuration from the programmer and is inefficient, while [56] is an error-prone methodology. For these reasons,

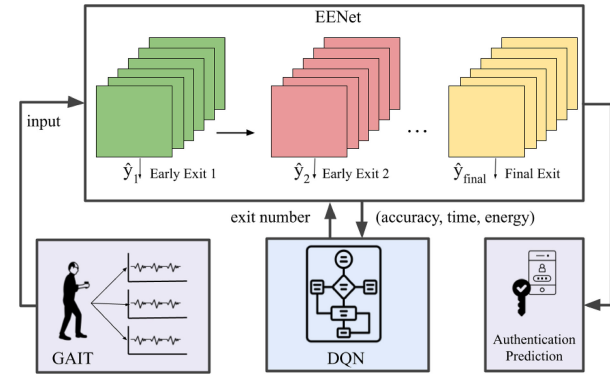


Fig. 1. Overview of our gait-based authentication method utilizing EENet and DQN.

we are implementing our own intermittent computing scheme, that is hardware-agnostic, versatile, and can be directly applied to any device and environment.

III. PROPOSED GAIT-BASED USER AUTHENTICATION

A. System Overview

In this section, we describe our proposed methodology on user authentication for lightweight IoT devices, using gait-based authentication with early exits and intermittent computations. The overview of our technique can be seen in Fig. 1. The input gait signal is captured by the embedded inertial sensors on the user's device and is sent to the EENet for inference. The EENet is configured to work in conjunction with the DQN. We can set the DQN to prioritize accuracy, inference execution time, or energy consumption and set the parameters of its reward function accordingly. Based on the input from the EENet and the reward function, it directs the EENet to take the most appropriate exit. The result of that process is the prediction of the EENet with the authorization decision. One major intention of this work is to ensure the compatibility and efficiency on devices that are disconnected from the electrical power grid. To avoid termination due to exhaustion of the IoT device's power source, we introduce an intermittent computations technique that can be deployed on any IoT device and perform gait-based authentication efficiently.

The remainder of this section includes more details on the input signal with the gait data in Section III-B, how the EENet framework is trained in Section III-C, the structure of our lightweight gait authentication CNN called EENet-lite in Section III-D, the configuration of our DQN reinforcement learning method in Section III-E, and our intermittent computation methodology in Section III-F.

B. Gait Signal

In order to use gait as an input signal, one needs to determine how to measure a user's walking form. This is very important because it is required to extract the gait signals efficiently with low overheads and low complexity. In this work, we consider six measurements: the linear acceleration on the x -, y -, and z -axes and the gyroscopic acceleration, which

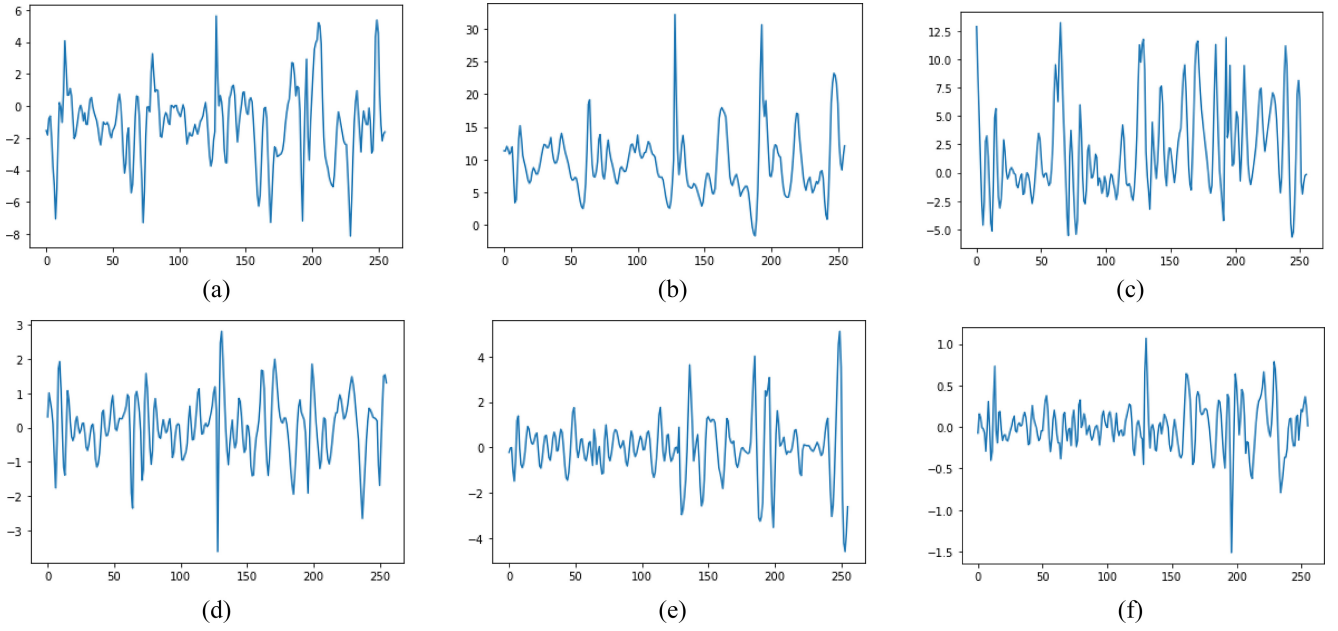


Fig. 2. Single-user gait input example displaying: (a) acc_x , (b) acc_y , (c) acc_z , (d) gyr_x , (e) gyr_y , and (f) gyr_z .

is the rotation rate around the x -, y -, and z -axes to represent the gait signal. We use linear acceleration instead of absolute acceleration, as it allows to disregard the effect of gravity on the measurements. Gravitational force adds unwanted noise to the measurements, since it is always present, with only small differences depending on altitude and latitude and does not contribute to identifying the user by their gait. Therefore, one can ignore absolute acceleration in favor of linear acceleration.

These gait measurements can be extracted from lightweight embedded sensors such as MPU9250 [57] that can be found on smartphones, tablets, smartwatches, and fitness trackers. For instance, a single MPU9250 sensor can collect all the necessary data with minimal consumption (3.4 mA). We adopt the preprocessing methodology presented in [19] for gait collection. The result is a tuple of six elements: three for linear acceleration and three for gyroscopic acceleration as mentioned. Each element is composed of 256 samples. More specifically, each sample is collected by identifying two continuous steps. Each step is interpolated to a fixed length of 128 samples using the linear interpolation function and we use a one-step overlap in order to increase the sample set efficiently. Based on the above, the preprocessing overhead requires only a small amount of basic operations and it is negligible. An example of the gait input signal is shown in Fig. 2.

C. Early Exit Neural Networks

The gait signals that are captured by the inertial sensors are used as an input for the user authentication. We introduce the use of EENets to process the incoming gait signals. Compared to traditional CNNs, the use of EENets reduces the total amount of total computations and multiply-and-accumulate (MAC) operations. This is achieved by configuring a number of early-exits inserted within the CNN of an EENet. The CNN

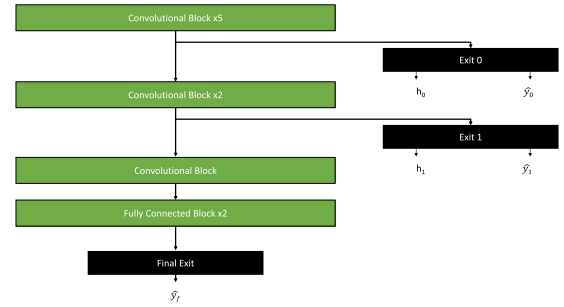


Fig. 3. Architecture of EENet-Lite, our proposed lightweight gait-based authentication model.

takes the earliest exit for which it reaches a certain level of confidence that its classification decision is correct. As a result, the CNN may not need to run in its entirety in order to produce a prediction. Hence, there is a tradeoff between performance and computational cost, making the deployment of EENets suitable on small- and medium-scale devices with restricted resources.

A variable number of early exits can be configured and their location in the CNN can be determined heuristically or it can be determined by an algorithm. The number and location of the early exits largely depend on the size and architecture of the CNN. In general, a CNN with n convolutional layers can have a maximum of m early exits, where $m \leq n$ at all times.

Every early exit has two distinct branches: 1) a *classification prediction branch* that uses a softmax activation function and 2) a *confidence branch* that uses a sigmoid activation function, as shown in Fig. 3. The output of the classification prediction branch is the prediction to the classification problem at that particular exit, whereas the output of the confidence branch is the confidence score h_i for exiting at that exit. If the confidence score is higher than the confidence threshold T , then this

early exit is taken and the algorithm stops running. Otherwise, the algorithm continues until it reaches an early exit, where $h_i > T$, or it reaches the final exit, whichever comes first.

1) *Training of EENet*: The training of the EENet is achieved by traditional backpropagation, with ADAM optimization. The model trains both the prediction and confidence branches independently for each early exit. The loss function that is used for the backpropagation during the training takes both prediction and cost into account and is defined as

$$\mathcal{L} = \mathcal{L}_P + \lambda \mathcal{L}_C \quad (1)$$

where $\mathcal{L}_P$ represents the loss from classification prediction and $\mathcal{L}_C$ represents the loss from computational cost. To calculate the classification loss, we consider the prediction vector that is defined as follows:

$$\hat{Y}_0 = h_0 \times \hat{y}_0 + (1 - h_0)[h_1 \times \hat{y}_1 + (1 - h_1)[\dots h_N \times \hat{y}_N + (1 - h_N) \times \hat{y}_{\text{final}}] \dots] \quad (2)$$

$$\hat{Y}_1 = h_1 \times \hat{y}_1 + (1 - h_1)[\dots h_N \times \hat{y}_N + (1 - h_N) \times \hat{y}_{\text{final}}] \dots] \quad (3)$$

$$\hat{Y}_N = h_N \times \hat{y}_N + (1 - h_N) \times \hat{y}_{\text{final}} \quad (4)$$

$$\hat{Y}_{\text{final}} = \hat{y}_{\text{final}}. \quad (5)$$

$\hat{Y}_i$ represents the output prediction vector produced by all early exits between i and N . $\hat{y}_i$ is the individual prediction for each exit and h_i is the confidence for each respective prediction. Once we obtain the values from (2)–(5), we use them to calculate the partial loss for classification according to

$$\mathcal{L}_P = - \sum_{k=1}^K y_k \times \log(\hat{Y}) \quad (6)$$

where K is the number of classes that the model classifies for. We get the respective cost values ($C_0, C_1, \dots, C_N, C_{\text{final}}$) the same way as described in (2)–(5) solving for cost instead of predicted value. Once we have the cost values, they are equal to the loss from computational cost

$$\mathcal{L}_C = C. \quad (7)$$

Thus, using (1), (6), and (7), one can obtain the value for the loss function for each early exit that is used for backpropagation.

2) *Inference With EENet*: Once training is complete, the EENet is ready to perform inference and generate predictions for each input signal it receives. Each early exit i produces a confidence score h_i and if $h_i > T$, where T is the confidence threshold, then that exit is taken and the local prediction $\hat{y}_i$ becomes the final classification decision $\hat{y}$. The confidence threshold T is set statically and it is the same across all early exits.

D. Design of EENet-Lite for IoT Devices

1) *Motivation*: In Section III-C, we presented the basic principles of EENets and why they are an appropriate and efficient architecture that achieves high accuracy, while significantly reducing the computational cost of complex classification problems. The EENet framework can be applied to

TABLE II
DETAILS OF THE CNN STRUCTURE

Layer	In channels	Out channels	Kernel size	Stride	Padding
Conv 1	1	2	3		1
Conv 2	2	4	3	(1,2)	1
Conv 3	4	8	3	1	1
Conv 4	8	16	3	1	2
Conv 5	16	16	3	1	2
Conv 6	16	32	3	1	2
Conv 7	32	64	3	1	1
Conv 8	64	64	3	1	1

several DNNs, such as ResNets [58], which are very effective classification algorithms. Most classification algorithms, however, have been designed to process images or video. The shape of such inputs is different from the input that we use for gait-based authentication. We want to be able to extract the maximum amount of features from the input as described in Section III-B, with the minimum number of computations and in the smallest number of layers. For that reason, we developed EENet-lite, a custom gait-based authorization algorithm for lightweight IoT devices.

2) *Architecture*: Our lightweight EENet algorithm needs to be small enough to run on hardware without parallelization support. Therefore, it should not have a large total number of convolutional layers, so that it can efficiently run even when no early exit is taken and the algorithm runs to the end. It also needs to be able to produce sufficiently good accuracy even when the first early exit is taken. As a result, it does not add value to configure a large amount of early exits, for instance, to have an early exit after each layer. To satisfy these conditions for EENet-lite, we have chosen to implement a total of eight convolutional layers, followed by two fully connected layers. The structure of the convolutional layers is shown in Table II.

Each convolutional layer is combined with a batch normalization layer and a ReLU activation function to form a convolutional block. Given the number of layers on EENet-lite, we implemented two early exits. The first exit can be taken after the fifth convolutional block and the second can be taken after the seventh convolutional block, according to Fig. 3.

E. Reinforcement Learning With DQN for EENet

As discussed in Section III-C, the confidence threshold is set statically for the EENet. Classification performance can vary, however, depending on the application, the dataset used or the type of input. That can potentially make the static threshold inefficient and affect the performance-efficiency tradeoff. We address this challenge, by introducing a method for dynamic exit selection. The dynamic exit selection is not a classification or a clustering problem. Rather, depending on the EENet state (input, accuracy, time, computations executed by EENet, and energy required for authentication), an action (exit selection) will be triggered. Considering the ability to perform such computations, we deploy reinforcement learning, to determine the best possible exit selection during runtime.

More specifically, we will use quality reinforcement learning (Q -learning) in this work. Q -Learning is a model-free

Algorithm 1 DQN Overview**Input:** Input: (accuracy, time, MAC ops, energy)**Output:** Output: exit number (n)

```

1:  $state \leftarrow (0, 0, 0, 0)$   $\triangleright$  initialization
2:  $b \leftarrow 0$ 
3: while  $b \times b_{size} \leq dataset_{size}$  do
4:    $exit_{no} \leftarrow n$   $\triangleright$  exit number is sent to eenet
5:    $accuracy, time, MAC, energy \leftarrow input$ 
6:    $state \leftarrow (accuracy, time, MAC, energy)$ 
7:    $R \leftarrow f(accuracy, time, energy)$ 
8:    $b \leftarrow b + 1$ 
9: end while

```

stochastic algorithm that can identify optimal action selection. However, due to the convergence issues of traditional Q -learning, we employ deep Q -learning network (DQN) in this work. The use of DQN removes correlations in the observation sequence and smooths the data distribution, addressing the converging issues that traditional Q -Learning may face. Given a set of states S and a set of actions A per state, when an action $a \in A$ is taken, the reinforcement learning agent transitions from state to state. The algorithm has a function that calculates the quality of each state–action combination, assigning a real number to it by using a reward function

$$Q : A \times S \rightarrow \mathbb{R}. \quad (8)$$

The DQN keeps getting updated every time that inference is run. By calculating and learning the reward value, DQN learns how to make decisions about the best possible next action and it gets trained to make the best decision possible in every subsequent situation. We ensure that the DQN we use is lightweight, so that it can work well in conjunction with the EENet, avoiding overheads.

Algorithm 1 outlines the employed DQN and its functionality based on the accuracy, inference time, # MAC operations, and energy consumption of the EENet. We choose these three parameters qualitatively, based on the nature of the application that we examine. Accuracy and inference time are important for authentication, whereas energy as well as inference time are important for the resource-restricted IoT device environment. Therefore, we focus on these three parameters to evaluate the performance of the DQN and retrain it, to improve its predictions. As a result, in our implementation of the DQN, the state is represented by a tuple that includes the values for accuracy, the execution time for inference, the number of MAC operations required for inference, and the energy that it takes to authenticate one sample. The action that moves the DQN agent from one state to the next is the exit that the EENet is directed to take. The DQN is a simple sequential NN. We chose that because it is smaller and quicker than other NNs, such as recurrent, residual, etc. We utilize two hidden layers of 24 neurons each. The activation functions are ReLU and we use the Adam optimizer. Table III includes an ablation study of a few different configurations for our DQN. We notice that for a relatively simple sequential NN the differences in accuracy and execution time are small. Based on that, we chose

TABLE III
DQN CONFIGURATION COMPARISON

Hidden Layers	No. of Neurons	Accuracy (%)	Inf. Time (ms)
2	24 x 24	85.66	2.699
2	24 x 12	85.33	3.401
3	24 x 24 x 12	85	3.384
2	24 x 6	86.38	3.211
2	12 x 12	85.13	3.664

to use the DQN that produced the smallest execution time and second-highest accuracy.

The reward function is expressed by the following equations:

$$R = \alpha \cdot R_a + \beta \cdot R_t + \gamma \cdot R_e \quad (9)$$

$$R_x = \begin{cases} r_{n+1}, & \text{if } x > \text{threshold}_n \\ r_n, & \text{if } \text{threshold}_{n-1} < x \leq \text{threshold}_n \\ \vdots & \vdots \\ r_0, & \text{otherwise} \end{cases} \quad (10)$$

where x stands for accuracy, inference time, or energy and R_x for R_a , R_t or R_e stand for the reward values for accuracy, inference time and energy consumption, respectively. In (9), α , β , and γ are scaling factors. The scaling factors allow us to determine the balance of the tradeoff between accuracy, execution time and energy consumption when we design the method and prioritize according to what is most important to our implementation.

In Algorithm 1, we present the function of the proposed DQN. After initialization (line 1), the algorithm works in batches (lines 2 and 3). For each batch, the DQN tells the EENet what action to take (what exit to take) (line 4). The EENet runs inference for that batch (line 5) and then sends the state in the form of the tuple of accuracy, time, MAC operations, and energy consumption back to the DQN (line 6). Then, the reward is calculated according to (9) and (10) (line 7), and then the DQN moves to the next batch. Based on its previous states and actions, the DQN is trained to dynamically make the optimal decision according to the reward function and learns to predict the next best exit from the EENet. At the end of the process, the DQN has worked with the EENet to run inference on the entire testing dataset and we can get the statistics for each element of the state tuple, along with the exit distribution from the EENet for all samples.

F. Intermittent Computing Through Checkpointing

The use of DQN for EENet addresses the challenge of dynamic exit selection and reducing the number of MAC operations performed without trading off the performance. However, one needs to note that medium- and small-scale IoT devices are meant to be versatile and easy to deploy in a variety of different environments. It also indicates that they are often deployed off the electrical grid, powered only by a battery or an energy harvester. This makes such IoT devices vulnerable to nontermination scenarios and power depletion attacks. For that reason, it is important to provide a framework that will allow all power-restricted devices to successfully complete the execution of the code that they run. In cases, when the energy available to them is not sufficient to run their code, provisions

need to be made to assure that the progress that they have made is saved. That way, when more energy becomes available, they can continue execution from the point where it was interrupted before. In order to achieve that we implement an intermittent computation technique.

Intermittent computation implementations are mostly applied to microcontrollers. They mostly utilize NVM, but there are also techniques that utilize volatile memory [41]. In either case, they are specific to the type of microcontroller and memory that they are developed for. That makes them very efficient, as they directly target specific memory locations to save and load checkpoints. That makes such implementations very efficient, but it also makes it difficult to adapt them to all other hardware.

In contrast, we introduce an intermittent computation implementation that is hardware agnostic and can run on any device where EENet-lite is deployed to run gait-based authentication. We also want the overhead of our technique to be as small as possible. For that reason, we implement static checkpoints, where we can store the output of certain layers on the system's DRAM, before it is passed on to the next convolutional layer. The reason that we use the DRAM to save the checkpoint is that it is much faster than saving information to the SD card that the Raspberry Pi uses as a main drive. We are taking advantage of the fact that DRAM is able to retain saved information for short amounts of time even in the case of loss of power [41]. There are implementations that use a stochastic approach for the placement of the checkpoints [56], but it is a less reliable approach. Additionally, given the small number of layers in EENet-lite, it is unnecessary to introduce the added overhead of the stochastic algorithm to determine the checkpoint locations.

We build checkpoints into the EENet-lite model so that we can avoid nontermination. The process that we employ is shown in Fig. 4. When the inference code that produces the authentication prediction starts, it first checks to see if the amount of energy that is available to the system E_{av} is equal or greater to the minimum amount of energy required to get to the next checkpoint or exit during authentication E_{min} (1). If that is the case, then it checks to see if there is a checkpoint saved or not (2). If not, it starts executing the EENet from the beginning (5). If there is a checkpoint stored, it gets loaded and the marker that points to the last saved location is checked (3). Then execution resumes from the last location where the checkpoint was taken (4). If at any point the EENet terminates, the process starts over (6). When the execution reaches a checkpoint location (7), a new checkpoint is taken (8). If there was another checkpoint already in place, the new one replaces the old one (8). Execution then continues until it reaches an exit that the EENet can take with confidence or the final exit. Then, the checkpoint gets deleted and an authentication prediction is produced.

Considering that our model CNN can take an early exit, it is desirable to make the device use one of the early exits and produce an authentication decision, as that ensures that a smaller number of computations is performed compared to running the entire NN. To inform our decision about the placement of the checkpoints, we consider the number of operations that need to be performed by layer, as shown in Table IV.

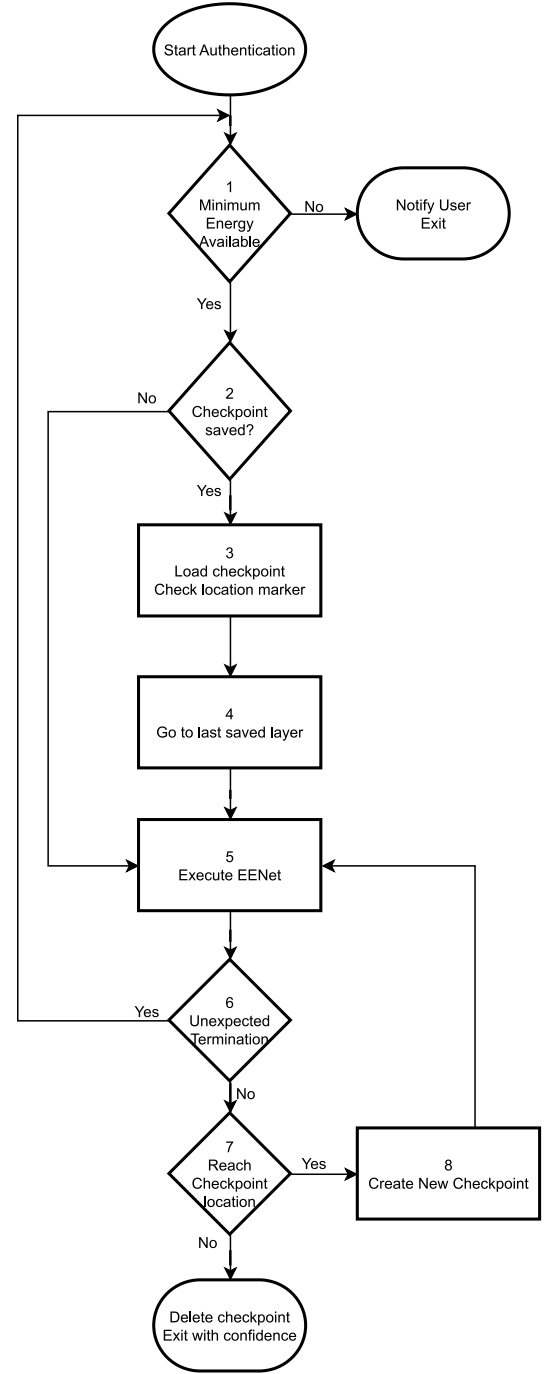


Fig. 4. Applying intermittent computations to our authentication scheme.

There is an early exit after layer Conv 5 and another early exit after layer Conv 7. In the interest of creating as balanced a distribution of MAC operations as possible going from one entry point to the code to its nearest checkpoint or exit, we consider

$$\text{MAC}(i) \simeq \text{MAC}(i + 1) \quad (11)$$

where $\text{MAC}(i)$ is the total number of MAC operations in i and

$$\begin{aligned} i \in [\text{start} \rightarrow \text{checkpoint1}, \text{checkpoint1} \rightarrow \text{early exit1}, \\ \text{early exit} \rightarrow \text{checkpoint2}, \text{checkpoint2} \rightarrow \text{early exit2}]. \end{aligned} \quad (12)$$

TABLE IV
EENET-LITE LAYER ANALYSIS

Layer	Input Shape	Output Shape	MAC	FLOPS
Conv 1	1x6x256	2x6x128	27,648	15,360
Conv 2	2x6x128	4x6x64	55,296	29,184
Conv 3	4x6x64	8x6x64	221,184	113,664
Conv 4	8x6x64	16x8x66	1,216,512	616,704
Conv 5	16x8x66	16x10x68	3,133,440	1,577,600
Conv 6	16x10x68	32x12x70	7,741,440	3,897,600
Conv 7	32x12x70	64x14x72	37,158,912	18,643,968
Conv 8	64x14x72	64x14x72	74,317,824	37,223,424
FC 1	64	20	2,540	1,280
FC 2	20	2	78	40

Algorithm 2 Creating Checkpoints

Input: State: output tensor from previous layer

Output: Current progress is saved

```

1: marker ← 0
2: while next checkpoint is reached do
3:   marker ← marker + 1
4:   checkpoint ← (marker, state)
5: end while
6: if exit is taken then
7:   delete checkpoint
8: end if

```

Having balanced $MAC(i)$ sections increases the likelihood that each checkpoint or exit is reached. As a result, it is less likely that there will be repeated instances when the device runs out of energy and shuts down. Every time a device runs out of energy, the computations it has performed—between the latest checkpoint and the point of sudden shutdown—are wasted and lost. Therefore, by limiting the times that there is a sudden shutdown, we reduce the total amount of wasted computations. By extension, we reduce the amount of wasted energy and minimize overhead.

The process of generating the checkpoints is described in Algorithm 2. EENet-lite starts running and once the checkpoint location is reached, the first checkpoint is saved. We use a marker (line 1) to keep track of the location where the state was saved and the checkpoint was created. After that, execution continues. If there are no interruptions, the authentication process (EENet) will keep running until the EENet reaches the next checkpoint or decides to take an exit, i.e., the authentication process is complete. In the case where the next checkpoint is reached, the marker is increased and the new checkpoint overwrites the previous one (lines 2–5). If an exit is taken, our EENet produces its output and the checkpoint is deleted (lines 6–8).

The process of loading checkpoints is described in Algorithm 3. Once the EENet starts executing, it checks to see if a checkpoint has been created. If so, it gets loaded (line 2) and the marker gets checked to determine where the checkpoint was made (line 3). Then, the tensor containing the last saved EENet state is loaded as an input to the next convolutional layer after the checkpoint was made (lines 4–6). If a checkpoint has not been previously created, execution starts from the beginning (lines 7–9).

Algorithm 3 Loading Checkpoints

Input: Authentication Request

Output: Execution resumes from last saved location

```

1: if Checkpoint exists then
2:   (marker, state) ← checkpoint
3:   Check marker
4:   Go to checkpoint location indicated by marker
5:   layeriinput ← state
6:   Execution resumes
7: else if No checkpoint exists then
8:   Start execution from the beginning
9: end if

```

IV. EXPERIMENTAL EVALUATION

A. Experimental Setup

We have run simulations to evaluate our methodology on two different environments: a PC with parallelization support and a medium-scale IoT device that runs on an ARM processor with no parallelization support. Our PC embeds an Intel i7 processor with 16 GB of RAM and an NVIDIA GeForce GTX1050 graphics card. Our IoT device is a Raspberry Pi 4B, with an ARM 64-bit processor and 8 GB of memory. For the setup, we used Ubuntu 22.04 LTS and set up a virtual environment with Python 3.9.12, Pip 22.1, Numpy 1.22.3, PyTorch and Torchvision 1.11.0, Scipy 1.8.0, and Tensorflow 2.8.0, built specifically to run on aarch64 architecture.

We compare the performance of the EENet framework to ResNets [58] as they have achieved high classification performance in a variety of applications and they serve as a good classification benchmark. We apply the EENet framework to ResNet18, ResNet34, and ResNet50, producing EENet18, EENet34, and EENet50, respectively. We will also compare all these models to our EENet-lite model. To determine how many early exits we should use for the ResNet-based EENets, we ran tests to see how the number of exits on EENet34 affected accuracy and performance. The results can be seen in Fig. 5. For the upcoming tests, we will consider five early exits. For the user authentication comparison, we will use the data from the whuGAIT dataset [19]. The training set consists of more than 65 000 samples, and the test set consists of 7600 samples.

B. Authentication Evaluation

To evaluate the performance of our authentication, we consider accuracy, inference time, and the number of MAC operations. Please note that the computations required for the softmax layer at each exit of the NN are included in the total amount of MAC operations. The exit distribution also tells us how many of the test samples were directed to take each exit. ResNets do not have any early exits built into them, while EENets has been configured with five possible early exits. Our lightweight EENet-lite model has been configured with two possible early exits, due to its small number of layers. We also show the effect of our DQN model on the performance of different models.

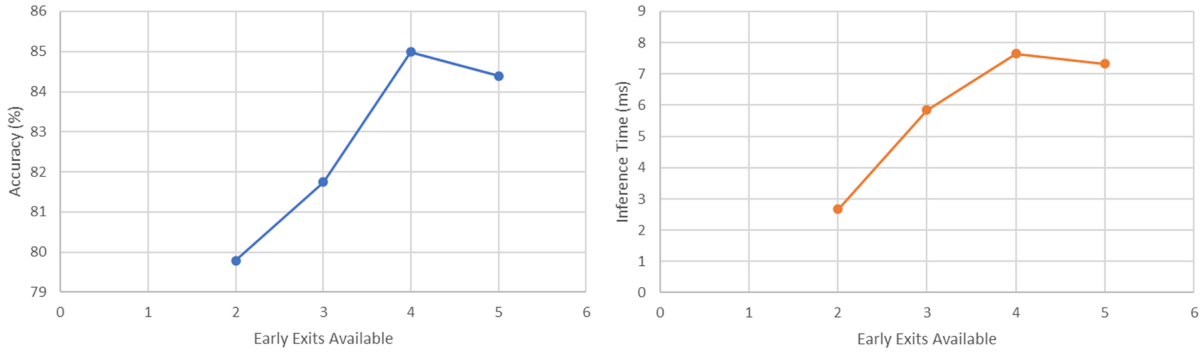


Fig. 5. Effect of number of available early exits in EENet34 to accuracy and inference time.

TABLE V
COMPARISON OF EENET VERSUS RESNET

Model	Accuracy	Exit Distribution	Inf. Time	Avg. #Ops
eenet-lite	74.37%	[7600,0,0]	2.815ms	4.78M
eenet-lite+dqn	85.66%	[2620,2800,2180]	2.699ms	56M
resnet18	91.49%	NA	7.354ms	137.89M
eenet18	79.70%	[0,1215,6332,53,0,0]	4.531ms	26.54M
eenet18+dqn	84.47%	[1210,1150,1230,1290,1400,1320]	5.763ms	49.06M
resnet34	91.71%	NA	12.741ms	279.67M
eenet34	84.36%	[0,0,3308,4292,0,0]	7.319ms	51.94M
eenet34+dqn	85.99%	[1190,940,1300,1470,1370,1330]	6.978ms	86.42M
resnet50	86.55%	NA	18.742ms	331.23M
eenet50	76.83%	[2,6181,1417,0,0,0]	7.023ms	46.9M
eenet50+dqn	85.99%	[990,1140,1230,1520,1410,1310]	9.737ms	97.28M

TABLE VI
COMPARISON OF EENET VERSUS BRANCHYNET

Model	Accuracy	Inf. Time
ResNet	80.70%	70.9ms
Branchy-ResNet [32]	79.17%	37.2ms
EENet	80.51%	12.1ms

The results of the performance are shown in Table V and inference times are marked based on our experiments using the PC environment. As expected, there is a tradeoff between performance and efficiency. The ResNets achieve higher accuracy scores by 9.6% on average from the EENet counterparts, while the EENets execute $2\times$ faster and need to perform $6\times$ fewer MAC operations than the ResNets. Some of the difference in accuracy on EENets can be made up for by the use of the DQN. The reward function of the DQN can evaluate previous predictions and improve performance dynamically. More specifically, applying the DQN improves accuracy by 8.5%, but it comes with $2.2\times$ more MAC operations and 0.87 ms slower inference time.

Table VI shows how the EENet framework compares to Branchynet [32], another methodology that utilizes early exits on a DNN. We evaluate both methodologies on the CIFAR10 dataset, and apply them on ResNet110 with two early exits, using a PC with GPU support.

As we can see, the EENet can achieve $3\times$ faster inference time than Branchynet, by taking computational cost into consideration for the calculation of the loss function. At the same time, it can achieve higher accuracy than Branchynet, comparable to the accuracy of the ResNet without early exit implementation.

TABLE VII
PERFORMANCE OF EENET-LITE ON PC VERSUS ARM

Model	Accuracy	Inf. Time (ARM)	Inf. Time (PC)	Avg. #Ops
eenet-lite exit1	74.37%	110.41ms	2.815ms	4.78M
eenet-lite exit2	75.24%	228.03ms	3.277ms	50.14M
eenet-lite+dqn	85.66%	254.12ms	2.699ms	56M
eenet34	84.36%	1,244.48ms	7.319ms	51.94M

TABLE VIII
ENERGY CONSUMPTION ON RASPBERRY PI

Model	Energy (mJ)	Time (ms)	Accuracy
eenet-lite exit1	505.489	110.41	74.37%
eenet-lite exit2	1,043.833	228.03	75.24%
eenet-lite+dqn	1,172.143	254.12	85.66%
eenet34	5,929.222	1,244.48	84.36%
eenet34+dqn	9,486.752	1,991.168	85.99%
resnet34	21,130.711	4,435.11	91.71%

Since it is important to create a methodology that works well on IoT devices with more limited resources, Table VII shows how EENet-lite performs on the Raspberry Pi compared to the PC in terms of inference time. Especially, because the Raspberry Pi uses its ARM processor with aarch64 architecture with no GPU support, its execution cannot be parallelized, and that is most evident when running EENet34. Running EENet-lite with DQN on the Raspberry Pi is $5\times$ faster than EENet34 while also achieving higher accuracy. We can also see the effect of using dynamically chosen exits compared to static exits. Utilizing the DQN increases the authentication performance by more than 10%, while only requiring 5.5% more operations than EENet-lite at the second exit. The difference between running on the ARM processor is that it takes on average $93\times$ longer to run a model on the Raspberry Pi, as expected. When we look at the difference of runtimes for EENet-lite with and without DQN, we notice that the presence of DQN has a different effect on the two platforms. That is, caused by the different processor architectures and especially the presence of GPU and parallelization support on the PC.

C. Energy Consumption

Energy consumption being important, we tested the energy required to run EENet on Raspberry Pi without the implementation of intermittent computing to get a baseline of the power cost. We tested EENet-lite for early exits 1 and 2, and with DQN support, as well as EENet34 with and without DQN support and ResNet34. Table VIII shows how they compare in

TABLE IX
MEMORY USAGE OF EENET

Model	Avg. Load-Store	Parameters Stored
eenet-lite exit1	4.006k	65.832k
eenet-lite exit2	27.430k	65.832k
eenet-lite+dqn	30.37k	65.832k
eenet50	652.1k	23.51M
eenet50+dqn	4.803M	23.51M

TABLE X
CHECKPOINT OVERHEAD ON PC

Checkpoint	Tensor	Avg Save	Median Save	Avg Load	Median Load
Conv1	2x6x128	2.534ms	1.192ms	5.545ms	5.985ms
Conv2	4x6x64	2.596ms	1.556ms	5.696ms	5.985ms
Conv3	8x6x64	2.857ms	1.644ms	7.129ms	6.946ms
Conv4	16x8x66	2.939ms	1.969ms	6.754ms	5.986ms
Conv6	32x12x70	2.024ms	1.661ms	6.716ms	5.238ms

terms of energy consumption and execution time. As expected, the number of operations required for each of these scenarios is reflected on the amount of energy required to complete said operations. It is also more pronounced on execution time, because of the lack of parallelization on smaller devices. The energy savings of taking an early exit are substantial. In general, the energy consumption of EENet-lite is $3.5\times$ to $41.8\times$ lower than ResNets.

D. Memory Analysis

Memory usage is also an important factor when designing techniques for IoT devices. Not only are there often restrictions in memory availability, but load-store operations can take a significant amount of time and energy to complete. Typical DNNs cause high memory usage. Table IX shows the memory requirements that EENets pose. EENets need to load only 4% or 27% of stored parameters when they run on their own or with DQN support, respectively. In the meantime, common ResNets load all of the stored parameters when they execute. Also, we notice that EENet-lite needs just $166.67\times$ – $357\times$ fewer parameters in comparison to ResNets or ResNet-based EENets. Memory requirements are specific to the model, so they would apply to any hardware that the NN would be deployed on.

E. Impact of Intermittent Computing

For the evaluation of our intermittent computing methodology, we analyze the overhead introduced with each save and load of the checkpoints. More specifically, we evaluate the impact of the checkpoint location in the CNN on the load and save time, due to the difference in the sizes of the output tensors. In Table X, we present the analysis for load and save times when the checkpoint is taken after various layers of the EENet-lite model. We notice that even though the size of the tensor changes, the difference in average and median time needed to save and load a checkpoint does not change proportionately. That is, probably owed to the fact that the size of the tensors is not just an integer number of words that get loaded all at once. So, increasing the size of the tensor does not linearly affect save and load times linearly.

Even though the save time increases as the tensor gets bigger, that changes after layer Conv6. Overall, the average

TABLE XI
CHECKPOINT OVERHEAD ON ARM

Checkpoint	Energy	Avg Save	Median Save	Avg Load	Median Load
Conv2	0.01697 J	2.439ms	1.799ms	1.267ms	0.961ms
Conv4	0.01867 J	2.794ms	2.070ms	1.283ms	1.007ms
Conv6	0.02691 J	4.310ms	3.217ms	1.568ms	1.184ms

save time varies by less than 0.915 ms. It is worth noting that the median is on average about 1 ms, which is about 38% less than the average. That suggests that for most of the samples, the save time is lower than the average save time. Similarly, load time gets higher after layer Conv3, but goes down again after layer Conv4. The average load time varies by 1.584 ms. After layers Conv1, Conv2, and Conv3, the difference between average and median load time is negligible, but for Conv4 and Conv6, we see that the median load time is lower than the average by more than 16.5%.

When using a medium-scale IoT device such as a Raspberry Pi, it is important to know not only the time that it takes to the checkpoint but also the energy that is required for each checkpoint to save and load. In Table XI, we see the overhead associated with the checkpointing process on the ARM processor of the Raspberry Pi. We notice that when the checkpoint takes place later in the CNN, the checkpoint save time is increased by as much as 77%, whereas the load time is affected less, getting increased by 24%. In both cases, we see again that the median values for the save and load time are considerably lower than the average, by 26% and 23%, respectively.

V. CONCLUSION

This work presents a novel user authentication methodology designed for deployment on IoT devices, using a user's gait as an input. The methodology employs an early-EENet, that allows the model to exit before completing all computations, when a certain level of confidence is achieved, maintaining high performance. Hence, it reduces the number of operations required for authentication, as well as memory and energy cost. We have further enhanced the design with a dynamic exit selection technique, based on a DQN, allowing the system to change its exit selection during runtime based on additional parameters (such as energy availability). This technique allows the design to have a more dynamic behavior while boosting accuracy. We evaluated EENets on Raspberry Pi and achieved $6\times$ fewer computations and $2\times$ lower inference time than ResNets. EENet-lite with DQN has accuracy of more than 85% on the WhuGAIT dataset. We have also implemented a software-based, hardware-agnostic methodology for intermittent computing, that allows our NN to create checkpoints at two different locations in EENet-lite. This way, we have created a solution that can be applied on a variety of platforms, even when there is limited energy available, as partial progress for an authentication request can be saved and loaded if needed. As a result, execution can resume and computations are less likely to get lost, further reducing the energy cost and mitigating the risk for power depletion attacks.

ACKNOWLEDGMENT

The views, opinions, and/or findings contained in this article are those of the author(s) and should not be interpreted as representing the official views or policies, either expressed or implied, of the Defense Advanced Research Projects Agency or the Department of Defense.

REFERENCES

- [1] B. Emmerson, "M2M: The Internet of 50 billion devices," *WinWin Mag.*, vol. 1, pp. 19–22, Jan. 2010.
- [2] O. Hersent, D. Boswarthick, and O. Elloumi, *The Internet of Things: Key Applications and Protocols*. Hoboken, NJ, USA: Wiley, 2011.
- [3] D. Miorandi, S. Sicari, F. De Pellegrini, and I. Chlamtac, "Internet of Things: Vision, applications and research challenges," *Ad Hoc Netw.*, vol. 10, no. 7, pp. 1497–1516, 2012.
- [4] R. Roman, J. Zhou, and J. Lopez, "On the features and challenges of security and privacy in distributed Internet of Things," *Comput. Netw.*, vol. 57, no. 10, pp. 2266–2279, 2013.
- [5] S. Sicari, A. Rizzardi, L. Grieco, and A. Coen-Porisini, "Security, privacy and trust in Internet of Things: The road ahead," *Comput. Netw.*, vol. 76, pp. 146–164, Jan. 2015. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1389128614003971>
- [6] H. Mikko and L. Nyman, "The Internet of (vulnerable) Things: On hypponen's law, security engineering, and IoT legislation," *Technol. Innov. Manage. Rev.*, vol. 7, no. 4, pp. 5–11, 2017.
- [7] M. Conti, A. Dehghantanha, K. Franke, and S. Watson, "Internet of Things security and forensics: Challenges and opportunities," vol. 78, pp. 544–546, Jan. 2018.
- [8] M. Patton, E. Gross, R. Chinn, S. Forbis, L. Walker, and H. Chen, "Uninvited connections: A study of vulnerable devices on the Internet of Things (IoT)," in *Proc. IEEE Joint Intell. Security Inform. Conf.*, 2014, pp. 232–235.
- [9] G. S. Singh and Supriya, "A study of encryption algorithms (RSA, DES, 3DES and AES) for information security," *Int. J. Comput. Appl.*, vol. 67, no. 19, pp. 33–38, 2013.
- [10] D. E. Denning, "Digital signatures with RSA and other public-key Cryptosystems," *Commun. ACM*, vol. 27, no. 4, pp. 388–392, Apr. 1984. [Online]. Available: <https://doi.org/10.1145/358027.358052>
- [11] X. Zhou and X. Tang, "Research and implementation of RSA algorithm for encryption and decryption," in *Proc. 6th Int. Forum Strategic Technol.*, vol. 2, 2011, pp. 1118–1121.
- [12] H. M. Wood, "The use of passwords for controlling access to remote computer systems and services," in *Manag. Requirem. Knowl. Int. Workshop*. Los Alamitos, CA, USA, Jun. 1977, pp. 27–33. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/AFIPS.1977.124>
- [13] F. Aloul, S. Zahidi, and W. El-Hajj, "Two factor authentication using mobile phones," in *Proc. IEEE/ACS Int. Conf. Comput. Syst. Appl.*, 2009, pp. 641–644.
- [14] J. Galka, M. Masior, and M. Salasa, "Voice authentication embedded solution for secured access control," *IEEE Trans. Consum. Electron.*, vol. 60, no. 4, pp. 653–661, Nov. 2014.
- [15] S. R. Borra, G. J. Reddy, and E. S. Reddy, "A broad survey on fingerprint recognition systems," in *Proc. Int. Conf. Wireless Commun. Signal Process. Netw. (WiSPNET)*, 2016, pp. 1428–1434.
- [16] S. Thavalengal, P. Bigioi, and P. Corcoran, "Iris authentication in handheld devices - considerations for constraint-free acquisition," *IEEE Trans. Consum. Electron.*, vol. 61, no. 2, pp. 245–253, May 2015.
- [17] J. Srinivas, A. K. Das, N. Kumar, and J. J. P. C. Rodrigues, "Cloud centric authentication for wearable Healthcare monitoring system," *IEEE Trans. Depend. Secure Comput.*, vol. 17, no. 5, pp. 942–956, Sep./Oct. 2020.
- [18] F. Sabahi, "Cloud computing security threats and responses," in *Proc. IEEE 3rd Int. Conf. Commun. Softw. Netw.*, 2011, pp. 245–249.
- [19] Q. Zou, Y. Wang, Y. Zhao, Q. Wang, C. Shen, and Q. Li, "Deep learning based gait recognition using smartphones in the wild," 2018, *arXiv:1811.00338*.
- [20] G. Cola, M. Avvenuti, A. Vecchio, G.-Z. Yang, and B. Lo, "An unsupervised approach for gait-based authentication," in *Proc. IEEE 12th Int. Conf. Wearable Implant. Body Sensor Netw. (BSN)*, 2015, pp. 1–6.
- [21] P. Zouridakis and S. M. P. Dinakarrao, "Performance-aware lightweight dynamic early-exit-based gait authentication," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, 2022, pp. 466–470.
- [22] A. Kale et al., "Identification of humans using gait," *IEEE Trans. Image Process.*, vol. 13, no. 9, pp. 1163–1173, Sep. 2004.
- [23] D. Baumfeld et al., "Reliability of Baropodometry on the evaluation of plantar load distribution: A transversal study," *BioMed Res. Int.*, vol. 2017, pp. 1–4, Mar. 2017.
- [24] G. Cola, M. Avvenuti, F. Musso, and A. Vecchio, "Gait-based authentication using a wrist-worn device," in *Proc. 13th Int. Conf. Mobile Ubiquitous Syst. Comput. Netw. Services*, New York, NY, USA, 2016, pp. 208–217. [Online]. Available: <https://doi.org/10.1145/2994374.2994393>
- [25] H. Li, A. Kadav, I. Durdanovic, H. Samet, and H. P. Graf, "Pruning filters for efficient ConvNets," 2017, *arXiv:1608.08710*.
- [26] H. Li, S. De, Z. Xu, C. Studer, H. Samet, and T. Goldstein, "Training quantized nets: A deeper understanding," 2017, *arXiv:1706.02379*.
- [27] M. Wess, S. M. P. Dinakarrao, and A. Jantsch, "Weighted quantization-regularization in DNNs for weight memory minimization toward HW implementation," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 37, no. 11, pp. 2929–2939, Nov. 2018.
- [28] P. R. Sutradhar et al., "Look-up-table based processing-in-memory architecture with programmable precision-scaling for deep learning applications," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 2, pp. 263–275, Feb. 2022.
- [29] P. D. S. Manoj et al., "A scalable network-on-chip microprocessor with 2.5D integrated memory and accelerator," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 64, no. 6, pp. 1432–1443, Jun. 2017.
- [30] K. Berestizshevsky and G. Even, "Sacrificing accuracy for reduced computation: Cascaded inference based on Softmax confidence," 2019, *arXiv:1805.10982*.
- [31] P. Panda, A. Sengupta, and K. Roy, "Conditional deep learning for energy-efficient and enhanced pattern recognition," in *Proc. Design Autom. Test Europe Conf. Exhibit. (DATE)*, 2016, pp. 475–480.
- [32] S. Teerapittayanon, B. McDanel, and H. Kung, "BranchyNet: Fast inference via early exiting from deep neural networks," in *Proc. 23rd Int. Conf. Pattern Recognit. (ICPR)*, 2016, pp. 2464–2469.
- [33] M. Figurnov et al., "Spatially adaptive computation time for residual networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2017, pp. 1790–1799.
- [34] Z. Wu et al., "BlockDrop: Dynamic inference paths in residual networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 8817–8826.
- [35] A. Veit and S. J. Belongie, "Convolutional networks with adaptive inference graphs," *Int. J. Comput. Vis.*, vol. 128, pp. 730–741, Mar. 2020.
- [36] X. Wang, F. Yu, Z.-Y. Dou, and J. Gonzalez, "SkipNet: Learning dynamic routing in convolutional networks," 2018, *arXiv:1711.09485*.
- [37] P. Zouridakis, "Software for performance- and energy-aware gait-based user authentication for IoT devices." Aug. 2021. [Online]. Available: <https://github.com/p-z-b-d/gait-based-authentication-eenet>
- [38] K. Maeng, A. Colin, and B. Lucia, "Alpaca: Intermittent execution without checkpoints," *Proc. ACM Program. Lang.*, vol. 1, p. 96, Oct. 2017. [Online]. Available: <https://doi.org/10.1145/3133920>
- [39] J. Van Der Woude and M. Hicks, "Intermittent computation without hardware support or programmer intervention," in *Proc. 12th USENIX Conf. Operat. Syst. Design Implement.*, 2016, pp. 17–32.
- [40] K. Maeng and B. Lucia, "Adaptive dynamic checkpointing for safe efficient intermittent computing," in *Proc. 13th {USENIX} Symp. Operat. Syst. Design Implement. ({OSDI})*, 2018, pp. 129–144.
- [41] H. Williams, X. Jian, and M. Hicks, "Forget failure: Exploiting SRAM data remanence for low-overhead intermittent computation," in *Proc. 25th Int. Conf. Architect. Support Program. Lang. Operat. Syst.*, New York, NY, USA, 2020, pp. 69–84. [Online]. Available: <https://doi.org/10.1145/3373376.3378478>
- [42] S. Ahmed, N. A. Bhatti, M. H. Siddiqui, and L. Mottola, "Efficient intermittent computing with differential checkpointing," in *Proc. 20th ACM SIGPLAN/SIGBED Int. Conf. Lang. Compilers Tools Embedded Syst.*, New York, NY, USA, 2019, pp. 70–81. [Online]. Available: <https://doi.org/10.1145/3316482.3326357>
- [43] E. Demir, "Early-exit convolutional neural networks," M.S. thesis, Graduate School Nat. Appl. Sci., Middle East Tech. Univ., Ankara, Turkey, 2019.
- [44] A. Gupta, M. Tripathi, T. J. Shaikh, and A. Sharma, "A lightweight anonymous user authentication and key establishment scheme for wearable devices," *Comput. Netw.*, vol. 149, pp. 29–42, Feb. 2019. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1389128618304389>

- [45] U. Banerjee, C. Juvekar, A. Wright, A. Arvind, and A. P. Chandrakasan, "An energy-efficient reconfigurable DTLS cryptographic engine for end-to-end security in IoT applications," in *Proc. IEEE Int. Solid State Circuits Conf. (ISSCC)*, 2018, pp. 42–44.
- [46] "NIST lightweight cryptography competition." Accessed: Aug. 2022. [Online]. Available: <https://csrc.nist.gov/Projects/Lightweight-Cryptography>
- [47] C. Beierle et al., "Lightweight AEAD and hashing using the sparkle permutation family," *IACR Trans. Symmetr. Cryptol.*, vol. 2020, no. S1, pp. 208–261, Jun. 2020. [Online]. Available: <https://tosc.iacr.org/index.php/ToSC/article/view/8627>
- [48] T. Iwata, M. Khairallah, K. Minematsu, and T. Peyrin, "Duel of the titans: The Romulus and Remus families of lightweight AEAD algorithms," *IACR Trans. Symmetr. Cryptol.*, vol. 2020, no. 1, pp. 43–120, May 2020. [Online]. Available: <https://tosc.iacr.org/index.php/ToSC/article/view/8560>
- [49] J. Daemen, S. Hoffert, M. Peeters, G. Van Assche, and R. Van Keer, "Xoodoo, a lightweight cryptographic scheme," *IACR Trans. Symmetr. Cryptol.*, vol. 2020, no. S1, pp. 60–87, Jun. 2020. [Online]. Available: <https://tosc.iacr.org/index.php/ToSC/article/view/8618>
- [50] G. Chen, W. Choi, X. Yu, T. Han, and M. Chandraker, "Learning efficient object detection models with knowledge distillation," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, Red Hook, NY, USA, 2017, pp. 742–751.
- [51] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," 2015, *arXiv:1503.02531*.
- [52] Y. Cheng, F. X. Yu, R. S. Feris, S. Kumar, A. Choudhary, and S.-F. Chang, "An exploration of parameter redundancy in deep networks with circulant projections," in *Proc. IEEE Int. Conf. Comput. Vis. (ICCV)*, 2015, pp. 2857–2865.
- [53] Y. Wu, Z. Wang, Z. Jia, Y. Shi, and J. Hu, "Intermittent inference with Nonuniformly compressed multi-exit neural network for energy harvesting powered devices," in *Proc. 57th ACM/EDAC/IEEE Design Autom. Conf.*, 2020, pp. 1–6.
- [54] B. Islam and S. Nirjon, "Zygarde: Time-sensitive on-device deep inference and adaptation on intermittently-powered systems," *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.*, vol. 4, no. 3, p. 82, Sep. 2020. [Online]. Available: <https://doi.org/10.1145/3411808>
- [55] A. Montanari, M. Sharma, D. Jenkus, M. Alloulah, L. Qendro, and F. Kawsar, "ePerceptive: Energy reactive embedded intelligence for Batteryless sensors," in *Proc. 18th Conf. Embedded Netw. Sensor Syst.*, New York, NY, USA, 2020, pp. 382–394. [Online]. Available: <https://doi.org/10.1145/3384419.3430782>
- [56] A. Colin and B. Lucia, "Termination checking and task decomposition for task-based intermittent programs," in *Proc. 27th Int. Conf. Compiler Construction*, New York, NY, USA, 2018, pp. 116–127. [Online]. Available: <https://doi.org/10.1145/3178372.3179525>
- [57] "MPU9250 data sheet." Accessed: Aug. 2022. [Online]. Available: <https://invensense.tdk.com/wp-content/uploads/2015/02/PS-MPU-9250A-01--v1.1.pdf>
- [58] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 770–778.



Pavlos Zouridakis (Graduate Student Member, IEEE) received the Bachelor of Science and Master of Science degrees in electrical and computers engineering from the Aristotle University of Thessaloniki, Thessaloniki, Greece, in 2009, and the Master of Science degree in computer engineering with emphasis on network security from George Mason University, Fairfax, VA, USA, in 2014, where he is currently pursuing the Ph.D. degree in electrical and computer engineering.



Sai Manoj Pudukotai Dinakarrao (Member, IEEE) received the Ph.D. degree in electrical and electronics engineering from Nanyang Technological University, Singapore, in 2015.

He is an Assistant Professor with George Mason University (GMU), Fairfax, VA, USA. Prior to joining GMU as an Assistant Professor, he served as a Research Assistant Professor and a Postdoctoral Research Fellow with GMU. His research interests include on-chip hardware security, neuromorphic computing, adversarial machine learning, self-aware SoC design, image processing, and time-series analysis.